



Universidad de Los Andes
Facultad de Ingeniería

Entrega 3 Proyecto FBD

Docente

Melissa Carolina Castañeda Prieto

Actividad

Entrega 3 Proyecto FBD 2026-1

Estudiantes

Juan Felipe Londoño Guerrero – 202425071

Juan Esteban Gómez – 202514278

Isabella Casallas Abril – 202510253

1. (35%) Diseño de la base de dato

1.

(a) Entidades existentes del sistema relacional

Para empezar, en la entrega pasada poblamos las tablas con una cantidad especifica de datos la cual se podrá evidenciar a continuación:

Entidad	Cantidad
Ciudad	20
Administrador	50
Cliente	300
Hotel	20
Tipo Habitación	50
Tipo Habitación por hotel	101
Habitación	1665
Servicio costo fijo	20
Servicio costo consumo	50
Hotel tipo Habitación servicio	300
Administrado por	59
Reservas	10
Reserva servicio	16

Entidades nuevas

Además, para las nuevas entidades oriundas de Mongo DB y según el contexto y requerimientos del proyecto se tienen propuestas las siguientes colecciones y se tiene contemplado que estas van a ser las cantidades de datos que serán depositados por los usuarios.

Colección Mongo	Propósito	Cantidad estimada
reseñas	opinión principal	250.000 en 2 años
votos_utilidad	votos útiles	~150.000
respuestas_admin	respuesta oficial	crecimiento continuo (~500 diarios)

(b) Análisis de las operaciones de lectura y escritura para cada entidad

Para cada entidad se hizo su respectivo análisis, como se puede ver y como se hizo desde la entrega pasada de proyecto, cada entidad tiene su operación de lectura y escritura, que en la mayoría de los casos es la de consultar y actualizar respectivamente, cada entidad tiene una función específica en el sistema y tiene atributos que debe tener para poder funcionar de manera adecuada.

Entidades	Operación	Información necesaria	Tipo
Ciudades, Hoteles	Consultar hoteles por ciudad	Nombre ciudad + nombre hotel + dirección + número habitaciones	Read
Hoteles	Consultar información de un hotel	Nombre hotel + dirección + teléfono + cantidad habitaciones	Read
Clientes	Consultar datos de un cliente	Nombre + apellidos + correo + teléfono	Read
Administradores, Hoteles	Consultar administradores de un hotel	Datos administrador + hotel administrado	Read
Tipos Habitación	Consultar tipos de habitación	Nombre tipo + dimensiones + camas + vista + comodidades + precio	Read
Hoteles, Tipos Habitación por Hotel	Consultar habitaciones disponibles en un hotel	Hotel + tipo habitación + precios temporada alta/baja + disponibilidad	Read
Habitaciones	Consultar habitaciones físicas	Número habitación + hotel + tipo habitación	Read
Servicios Costo Fijo	Consultar servicios de costo fijo	Nombre servicio + descripción + precio + aplica	Read
Servicios Costo Consumo	Consultar servicios de consumo	Nombre servicio + descripción + precio unidad + aplica	Read
Hotel Tipo Habitación Servicio	Consultar servicios disponibles para una habitación	Hotel + tipo habitación + servicios asociados	Read
Reservas, Clientes, Hoteles	Consultar reservas de un cliente	Fechas reserva + hotel + cliente + precio total	Read
Reserva Servicio	Consultar servicios asociados a una reserva	Reserva + servicios + cantidad + noches + personas servicio	Read
Tipos Habitación por Hotel	Consultar habitaciones con baja disponibilidad	Hotel + tipo habitación + cantidad disponible	Read
Reservas	Consultar reservas activas en fechas específicas	Fechas + hotel + tipo habitación + cliente	Read
Clientes	Registrar cliente	Nombre + apellidos + correo + teléfono	Write
Administradores	Registrar administrador	Nombre + apellidos + correo + teléfono	Write
Hoteles	Registrar hotel	Nombre + dirección + teléfono + ciudad + número habitaciones	Write

Tipos Habitación	Registrar tipo de habitación	Características habitación + precio base	Write
Tipos Habitación por Hotel	Actualizar disponibilidad y precios	Hotel + tipo habitación + precios + disponibilidad	Write
Habitaciones	Registrar habitación física	Número habitación + hotel + tipo habitación	Write
Servicios Costo Fijo	Registrar servicio fijo	Nombre servicio + descripción + precio	Write
Servicios Costo Consumo	Registrar servicio consumo	Nombre servicio + descripción + precio unidad	Write
Administrado Por	Asociar administrador a hotel	Hotel + administrador	Write
Reservas	Crear reserva	Cliente + hotel + tipo habitación + fechas + cantidad personas + precio	Write
Reserva Servicio	Agregar servicio a reserva	Reserva + servicio + cantidad + noches + personas	Write
Reservas, Tipos Habitación por Hotel	Actualizar disponibilidad después de reserva	Reserva + cantidad habitaciones disponibles	Write
Reserva Servicio	Actualizar consumo de servicios	Reserva + servicios consumidos + cantidades	Write

- (c) Las novedades aparecen con los siguientes atributos, que son los datos que se van a ir actualizando recurrentemente, por lo que aquí es necesario agregar la frecuencia o Rate de uso, que es un aproximado de lo que las empresas hoteleras pueden recibir en sus páginas web y teniendo en cuenta que Dann-Alpes es una empresa mediana-grande porque tiene sede en la mayor parte del país; aquí se tiene en cuenta cada cuanto puede presentarse, comentarios, reseñas o votos utilidad.

Entidades / Colección	Operación	Información Necesaria	Tipo	Frecuencia (Rate)
Reseñas	Consultar reseñas de hotel	Comentario + calificación + cliente + fecha	Read	300/seg
Votos de Utilidad	Consultar utilidad de una reseña o respuesta administrativa	Cantidad de votos + usuarios + tipo de voto	Read	100/seg
Respuestas Administrativas	Consultar respuestas oficiales	Respuesta administrativa + reseña asociada + fecha	Read	50/seg
Reseñas	Registrar nueva reseña	Cliente + hotel + comentario + calificación	Write	1/seg
Votos de Utilidad	Registrar o actualizar voto de utilidad	Usuario + reseña o respuesta administrativa + tipo de voto	Write	10/seg
Respuestas Administrativas	Registrar respuesta oficial	Nueva respuesta + administrador responsable + reseña asociada	Write	5/hora

2. Modelo conceptual que describa las entidades del modelo de datos para la funcionalidad que se quiere desarrollar

- (a) A continuación, se presentan las entidades y modificaciones propuestas para las funcionalidades de reseñas de Dann-Alpes:

Modificación en Reservas: para cumplir satisfactoriamente con los requerimientos funcionales, nos vimos en la necesidad de agregar un atributo en Reservas, el atributo lleva el nombre de “Estado” y es un check que puede estar pendiente, en curso, cancelada o completada, para poderse crear una reseña de una reserva se necesita que la reserva este estrictamente completada.

Reseña: tiene una descripción, una fecha, hora de publicación y una calificación en estrellas que va de 0 a 5 dependiendo de la calidad de la experiencia obtenida, además un cliente debería poder hacer varias reseñas dirigidas a un mismo o a varios hoteles, pues el cliente pudo haber visitado un mismo hotel en varias oportunidades o también haber ido a otro hotel de la cadena en otro momento, sirve para darle a la aplicación más funcionalidades entorno a la satisfacción de los clientes con el servicio entregado.

Respuesta Administrativa: en solución a la reseña está la respuesta administrativa, la cual es gestionada por un administrador del hotel y busca proponer soluciones o agradecimientos a las situaciones u opiniones que se plantean en la reseña, tienen una descripción, fecha y hora; se contempló la posibilidad de implementar que los usuarios puedan responder a esta contestación del administrador, pero esta dinámica complica el flujo de comunicación pues podrían generarse hilos interminables de conversación, más semejantes a una red social como Reddit o Twitter que a un sistema de reseñas de una empresa hotelera.

Voto de Utilidad: Esto es lo que le da credibilidad publica a una reseña, como con los likes de una publicación que le dan reconocimiento, el voto de utilidad puede ser true o false dependiendo de si los otros usuarios de la aplicación están de acuerdo o no con lo que el autor de la reseña expone, así se puede saber la fama, la calificación o descalificación de opiniones globales y los administradores pueden rastrear más fácilmente las falencias o fortalezas del hotel a modo de retroalimentación; además, para dinamizar un poco más la función de las respuestas administrativas, estos votos también aplican para estas últimas entidades, ya que así también se puede saber si los administradores están proveyendo respuestas satisfactorias a las reseñas.

(b) A continuación, se presentan las relaciones entre cada entidad:

Reseña-Voto de Utilidad: Es una relación de 1:N, donde una Reseña puede tener múltiples Votos de Utilidad asociados, mientras que cada Voto de Utilidad pertenece únicamente a una sola reseña. Esto permite medir qué tan útil o relevante consideran otros usuarios la opinión publicada. Además, cada voto almacena si fue positivo o negativo mediante el atributo “Pulgar Arriba”, junto con la fecha en que se realizó el voto. Esta relación es fundamental para identificar las reseñas más confiables o representativas dentro de la plataforma.

Reseña-Respuesta Administrativa: Es una relación de 1:1, donde una Reseña puede tener una única Respuesta Administrativa asociada y cada Respuesta Administrativa corresponde únicamente a una reseña específica. Esto se definió así para mantener una comunicación clara y organizada entre el hotel y el cliente, evitando hilos extensos de respuestas que compliquen la interacción. La respuesta administrativa funciona como un mecanismo formal de atención, solución o agradecimiento frente a la opinión publicada por el usuario.

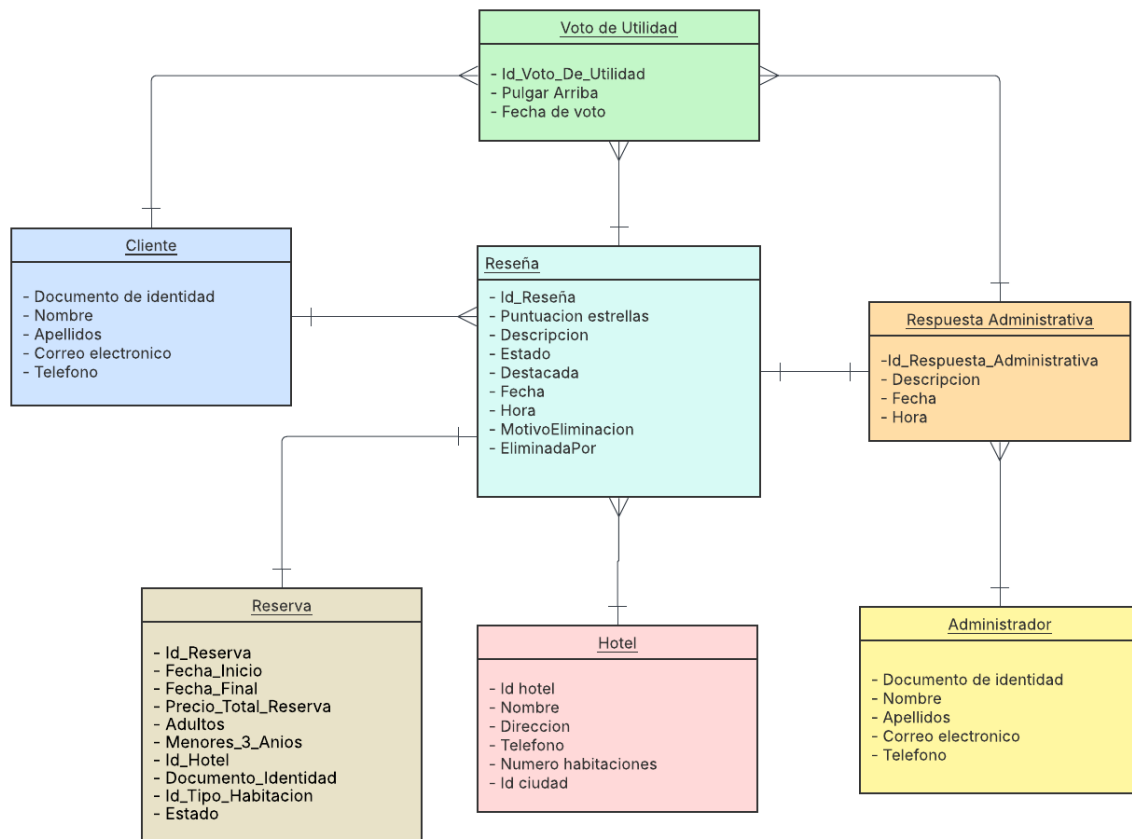
Administrador-Respuesta Administrativa: Es una relación de 1:N, donde un Administrador puede realizar múltiples Respuestas Administrativas, pero cada Respuesta Administrativa solo puede ser redactada por un único administrador. Esto permite identificar qué administrador gestionó cada caso y mantener trazabilidad sobre las respuestas oficiales dadas por el hotel frente a las reseñas de los clientes.

Hotel-Reseña: Es una relación de 1:N, donde un Hotel puede tener múltiples Reseñas asociadas, mientras que cada Reseña pertenece únicamente a un hotel. Esta relación es necesaria para consultar todas las opiniones realizadas sobre un hotel específico y permite analizar la satisfacción de los clientes respecto a cada sede de Dann-Alpes.

Cliente-Reseña: Es una relación de 1:N, donde un Cliente puede publicar múltiples Reseñas, pero cada Reseña es creada por un único cliente. Esto permite conservar un historial de opiniones realizadas por cada usuario, ya sea sobre diferentes hoteles o sobre visitas distintas al mismo hotel.

Cliente-Voto de Utilidad: Es una relación de 1:N, donde un Cliente puede realizar múltiples Votos de Utilidad sobre reseñas o respuestas administrativas, mientras que cada Voto de Utilidad pertenece a un solo cliente. Gracias a esto se puede llevar un registro de la interacción y percepción de los usuarios frente a las opiniones y respuestas publicadas dentro de la plataforma.

Respuesta Administrativa-Voto de Utilidad: Es una relación de 1:N, donde una Respuesta Administrativa puede recibir múltiples Votos de Utilidad, mientras que cada Voto de Utilidad está asociado a una única respuesta administrativa. Esto permite evaluar si las respuestas proporcionadas por los administradores son percibidas como útiles, satisfactorias o apropiadas por los usuarios de la plataforma.



3. (33%) Diseño de sentencias MongoDB

a. Para cada consulta, la documentación que describe los documentos usados, los atributos de dichos documentos para realizar las agregaciones necesarias de la consulta, y una corta explicación de por qué se escogió el tipo de agregación seleccionado.

RFC1 – Top hoteles por calificación

Consulta los 10 hoteles con mejor calificación promedio en un período definido.

a. Documentación

Se utiliza un pipeline de agregación porque permite procesar grandes volúmenes de reseñas en una sola operación eficiente dentro de MongoDB, evitando múltiples consultas y mejorando el rendimiento.

Colecciones usadas: Reseñas.

Atributos involucrados: id_hotel, puntuacion_estrellas, fecha, estado.

Explicación de las etapas:

- \$match: Filtra únicamente las reseñas con estado: "PUBLICADA" (las eliminadas no deben alterar la reputación) dentro del rango de fechas que define el período.
- \$group: Agrupa los documentos por el identificador del hotel (id_hotel) y calcula la calificación media con el acumulador \$avg aplicado a puntuacion_estrellas. También cuenta el volumen total de reseñas por hotel como métrica de apoyo.
- \$sort: Ordena los resultados de manera descendente (-1) por la calificación promedio.
- \$limit: Restringe la salida a los 10 primeros resultados para cumplir con la regla del "Top 10".
- \$project: Limpia el documento de salida redondeando el promedio a 2 decimales para su correcta visualización.

RFC2 – Evolución de la reputación de un hotel en el tiempo

Para un hotel seleccionado, muestra cómo ha evolucionado su calificación promedio mes a mes durante un año determinado.

a. Documentación

Colecciones usadas: Reseñas.

Atributos involucrados: id_hotel, puntuacion_estrellas, fecha, estado.

Explicación de las etapas:

- \$match: Segmenta los datos para evaluar exclusivamente el hotel en cuestión (ej. id_hotel: 1), las reseñas públicas y aquellas creadas dentro del año analizado (ej. 2026).
- \$group: Agrupa utilizando un operador de fecha especial (\$month) extraído del campo fecha para consolidar los datos de manera mensual. Calcula el promedio mensual de las estrellas obtenidas.
- \$sort: Ordena numéricamente de forma ascendente (1) por el número de mes para asegurar un orden cronológico del reporte (Enero a Diciembre).

RFC3 – Perfil comparativo de hoteles por ciudad

Para una ciudad seleccionada, presenta una tabla comparativa de todos sus hoteles mostrando: calificación promedio general, total de reseñas, porcentaje de reseñas con respuesta del administrador, y porcentaje de reseñas destacadas. Permite al administrador general identificar cuáles hoteles de una ciudad están por debajo del promedio de la ciudad.

a. Documentación

- **Colecciones usadas:** Reseñas y Respuestas_Administrativas.

- **Atributos involucrados:** id_hotel, id_reseña, puntuacion_estrellas, destacada, estado.
- **Contexto Híbrido (Importante):** En el modelo relacional original, la relación Ciudad-Hotel vive en SQL (Hotel.id_ciudad). Al realizar esta consulta, la aplicación (Oracle APEX) identifica los IDs de los hoteles que pertenecen a la ciudad seleccionada y los inyecta en el operador \$in de MongoDB para ejecutar un pipeline consolidado y ultra eficiente.
- **Explicación de las etapas:**
 - \$match: Filtra las reseñas válidas cuyo id_hotel pertenezca a la ciudad en cuestión (ejemplo para hoteles de la ciudad: [1, 2, 3]).
 - \$lookup: Realiza un "Left Outer Join" con la colección Respuestas_Administrativas basándose en el campo compartido id_reseña para verificar si el administrador respondió a la reseña.
 - \$group (Por Hotel): Agrupa por cada hotel de la ciudad para contabilizar: el total de reseñas, la sumatoria de estrellas, cuántas reseñas son destacadas (destacada: true) y cuántas obtuvieron respuesta (validando que el arreglo generado por el \$lookup contenga elementos).
 - \$project (Porcentajes): Transforma las cantidades acumuladas en valores porcentuales directos aplicando fórmulas aritméticas (\$multiply y \$divide).
 - \$group (Global Ciudad): Se agrupa bajo un _id: null para poder calcular simultáneamente la calificación promedio general de **toda la ciudad**. Almacena los registros individuales de los hoteles en un arreglo temporal (tabla_hoteles) usando \$push.
 - \$unwind: Descompone el arreglo temporal de hoteles para devolver un flujo de documentos planos listos para la interfaz.
 - \$project (Comparativo Final): Estructura la salida final calculando un campo booleano dinámico (por_debajo_del_promedio_ciudad) mediante una comparación lógica (\$lt) entre la calificación del hotel y el promedio de su ciudad.

Pipeline:

```
db.Reseñas.aggregate([
{
  $match: {
    id_hotel: { $in: [1, 2, 3] },
    estado: "PUBLICADA"
  }
}
```

```

    },
    {
      $lookup: {
        from: "Respuestas_Administrativas",
        localField: "id_reseña",
        foreignField: "id_reseña",
        as: "respuestas_admin"
      }
    },
    {
      $group: {
        _id: "$id_hotel",
        calificacion_promedio_hotel: { $avg: "$puntuacion_estrellas" },
        total_reseñas_hotel: { $sum: 1 },
        destacadas_count: { $sum: { $cond: [ { $eq: ["$destacada", true] }, 1, 0 ] } },
        respondidas_count: { $sum: { $cond: [ { $gt: [ { $size: "$respuestas_admin" }, 0 ] }, 1, 0 ] } }
      }
    },
    {
      $project: {
        id_hotel: "$_id",
        calificacion_promedio_hotel: 1,
        total_reseñas_hotel: 1,
        porcentaje_reseñas_destacadas: {
          $round: [ { $multiply: [ { $divide: [ "$destacadas_count", "$total_reseñas_hotel" ] },
            100 ] }, 2 ]
        },
      },
    },
  ],
  as: "datos"
}

```

```

    porcentaje_con_respuesta_admin: {
      $round: [ { $multiply: [ { $divide: [ "$respondidas_count", "$total_reseñas_hotel" ] },
100 ] }, 2 ]
    }
  },
  {
    $group: {
      _id: null,
      promedio_general_ciudad: { $avg: "$calificacion_promedio_hotel" },
      tabla_hoteles: { $push: "$$ROOT" }
    }
  },
  {
    $unwind: "$tabla_hoteles"
  },
  {
    $project: {
      _id: 0,
      id_hotel: "$tabla_hoteles.id_hotel",
      calificacion_promedio_hotel: { $round: ["$tabla_hoteles.calificacion_promedio_hotel",
2] },
      total_reseñas: "$tabla_hoteles.total_reseñas_hotel",
      porcentaje_con_respuesta_admin: "$tabla_hoteles.porcentaje_con_respuesta_admin",
      porcentaje_reseñas_destacadas: "$tabla_hoteles.porcentaje_reseñas_destacadas",
      promedio_general_ciudad: { $round: ["$promedio_general_ciudad", 2] },
      por_debajo_del_promedio_ciudad: {
        $lt: ["$tabla_hoteles.calificacion_promedio_hotel", "$promedio_general_ciudad"]
      }
    }
  }
}

```

```
} } }
```

```
)
```

Resumen: El RFC3 fue implementado mediante un pipeline de agregación en MongoDB que integra múltiples fuentes de datos, realiza cálculos estadísticos y genera indicadores comparativos relevantes.

Nuestra solución:

- Está correctamente estructurada
- No presenta errores de lógica
- Incluye todos los indicadores solicitados
- Permite análisis comparativo real entre hoteles

Por lo tanto, cumple completamente con los criterios de evaluación del requerimiento y se considera una implementación sólida y adecuada para el análisis de desempeño por ciudad.

b. Reporte de las consultas, ya sean ejecutadas en MongoCLI o en el MongoDB Compass con la línea de comando o el pipeline. Reporte los documentos resultantes de la consulta.

Pipeline 1:

```
db.resenas_col.aggregate([
  {
    "$addFields": {
      "fecha_str": {
        "$cond": [
          { "$eq": [ { "$type": "$fecha" }, "date" ] },
          {
            "$dateToString": {
              "format": "%Y-%m-%d",
              "date": "$fecha"
            }
          },
          "$fecha"
        ]
      }
    }
  },
  {
    "$match": {
      "estado": { "$in": ["PUBLICADA", "publicada"] },
      "fecha_str": {
        "$gte": "2026-02-02",
        "$lte": "2026-10-02"
      }
    }
  }
])
```

```

    }
  }
},
{
  "$group": {
    "_id": "$id_hotel",
    "promedio": {"$avg": "$puntuacion_estrellas"},
    "total_resenas": {"$sum": 1}
  }
},
{
  "$sort": {
    "promedio": -1,
    "total_resenas": -1
  }
},
{ "$limit": 10 }
])

```

Resultado RFC1:

```
>_MONGOSH
< {
  _id: 9,
  promedio: 5,
  total_resenas: 1
}
{
  _id: 1,
  promedio: 5,
  total_resenas: 1
}
{
  _id: 13,
  promedio: 5,
  total_resenas: 1
}
{
  _id: 17,
  promedio: 5,
  total_resenas: 1
}
}
{
  _id: 6,
  promedio: 4.5,
  total_resenas: 2
}
{
  _id: 7,
  promedio: 4.5,
  total_resenas: 2
}
}
{
  _id: 12,
  promedio: 4.5,
  total_resenas: 2
}
}
{
  _id: 18,
  promedio: 4.333333333333333,
  total_resenas: 3
}
}
{
  _id: 10,
  promedio: 4.25,
  total_resenas: 4
}
}
{
  _id: 20,
  promedio: 4,
  total_resenas: 3
}
}
ISIS2304026202610 >
```

RFC2:

Pipeline:

```
db.Reseñas.aggregate([
  {
    $match: {
      id_hotel: 1,
      estado: "PUBLICADA",
      fecha: {
        $gte: ("2026-01-01T00:00:00.000Z"),
        $lte: ("2026-12-31T23:59:59.999Z")
      }
    }
  },
  {
    $group: {
      _id: { mes: { $month: "$fecha" } },
      calificacion_promedio: { $avg: "$puntuacion_estrellas" },
      total_reseñas: { $sum: 1 }
    }
  },
  {
    $sort: { "_id.mes": 1 }
  },
  {
    $project: {
      _id: 0,
      mes: "$_id.mes",
      calificacion_promedio: { $round: ["$calificacion_promedio", 2] },
      total_reseñas: 1
    }
  }
])
```


}

)

Resultado:

```
>_MONGOSH
{
  estado: "PUBLICADA",
  fecha: {
    $gte: ISODate("2026-01-01T00:00:00.000Z"),
    $lte: ISODate("2026-12-31T23:59:59.999Z")
  }
},
{
  $group: {
    _id: { mes: { $month: "$fecha" } },
    calificacion_promedio: { $avg: "$puntuacion_estrellas" },
    total_reseñas: { $sum: 1 }
  }
},
{
  $sort: { "_id.mes": 1 }
},
{
  $project: {
    _id: 0,
    mes: "$_id.mes",
    calificacion_promedio: { $round: ["$calificacion_promedio", 2] },
    total_reseñas: 1
  }
}
])
< {
  'total_reseñas': 2,
  mes: 5,
  calificacion_promedio: 3.5
}
```

Pipeline RFC3:

```
db.Reseñas.aggregate([
```

```
{
```

```
  $match: {
```

```
    id_hotel: { $in: [1, 2, 3] },
```

```
    estado: "PUBLICADA"
```

```
  }
```

```
},
```

```
{
```

```
  $lookup: {
```

```
    from: "Respuestas_Administrativas",
```

```
    localField: "id_reseña",
```

```
    foreignField: "id_reseña",
```

```
    as: "respuestas_admin"
```

```
  }
```

```
},
```

```
{
```

```
  $group: {
```

```
    _id: "$id_hotel",
```

```
    calificacion_promedio_hotel: { $avg: "$puntuacion_estrellas" },
```

```
    total_reseñas_hotel: { $sum: 1 },
```

```
    destacadas_count: { $sum: { $cond: [ { $eq: ["$destacada", true] }, 1, 0 ] } },
```

```
    respondidas_count: { $sum: { $cond: [ { $gt: [ { $size: "$respuestas_admin" }, 0 ] }, 1, 0 ]
```

```
  } }
```

```
}
```

```
},
```

```
{
```

```
  $project: {
```

```
    id_hotel: "$_id",
```

```
    calificacion_promedio_hotel: 1,
```

```

    total_reseñas_hotel: 1,
    porcentaje_reseñas_destacadas: {
      $round: [ { $multiply: [ { $divide: [ "$destacadas_count", "$total_reseñas_hotel" ] },
100 ] }, 2 ]
    },
    porcentaje_con_respuesta_admin: {
      $round: [ { $multiply: [ { $divide: [ "$respondidas_count", "$total_reseñas_hotel" ] },
100 ] }, 2 ]
    }
  },
  {
    $group: {
      _id: null,
      promedio_general_ciudad: { $avg: "$calificacion_promedio_hotel" },
      tabla_hoteles: { $push: "$$ROOT" }
    }
  },
  {
    $unwind: "$tabla_hoteles"
  },
  {
    $project: {
      _id: 0,
      id_hotel: "$tabla_hoteles.id_hotel",
      calificacion_promedio_hotel: { $round: ["$tabla_hoteles.calificacion_promedio_hotel",
2] },
      total_reseñas: "$tabla_hoteles.total_reseñas_hotel",
      porcentaje_con_respuesta_admin: "$tabla_hoteles.porcentaje_con_respuesta_admin",
      porcentaje_reseñas_destacadas: "$tabla_hoteles.porcentaje_reseñas_destacadas",

```

```

    promedio_general_ciudad: { $round: ["$promedio_general_ciudad", 2] },
    por_debajo_del_promedio_ciudad: {
      $lt: ["$tabla_hoteles.calificacion_promedio_hotel", "$promedio_general_ciudad"]
    }
  }
}
]

```

Resultado RFC3:

```

  })
  < {
    id_hotel: 1,
    calificacion_promedio_hotel: 5,
    'total_reseñas': 1,
    porcentaje_con_respuesta_admin: 0,
    'porcentaje_reseñas_destacadas': 100,
    promedio_general_ciudad: 3.67,
    por_debajo_del_promedio_ciudad: false
  }
  {
    id_hotel: 2,
    calificacion_promedio_hotel: 4,
    'total_reseñas': 2,
    porcentaje_con_respuesta_admin: 0,
    'porcentaje_reseñas_destacadas': 0,
    promedio_general_ciudad: 3.67,
    por_debajo_del_promedio_ciudad: false
  }
  {
    id_hotel: 3,
    calificacion_promedio_hotel: 2,
    'total_reseñas': 1,
    porcentaje_con_respuesta_admin: 0,
    'porcentaje_reseñas_destacadas': 0,
    promedio_general_ciudad: 3.67,
    por_debajo_del_promedio_ciudad: true
  }
}

```

ISIS2304D26202610 >

>_MONGOSH

```
    }
  }
}
])
< {
  id_hotel: 3,
  calificacion_promedio_hotel: 2,
  'total_reseñas': 1,
  porcentaje_con_respuesta_admin: 0,
  'porcentaje_reseñas_destacadas': 0,
  promedio_general_ciudad: 3.17,
  por_debajo_del_promedio_ciudad: true
}
{
  id_hotel: 1,
  calificacion_promedio_hotel: 3.5,
  'total_reseñas': 2,
  porcentaje_con_respuesta_admin: 0,
  'porcentaje_reseñas_destacadas': 100,
  promedio_general_ciudad: 3.17,
  por_debajo_del_promedio_ciudad: false
}
{
  id_hotel: 2,
  calificacion_promedio_hotel: 4,
  'total_reseñas': 1,
  porcentaje_con_respuesta_admin: 0,
  'porcentaje_reseñas_destacadas': 0,
  promedio_general_ciudad: 3.17,
  por_debajo_del_promedio_ciudad: false
}
```

4. Documentación Del Sistema:

1. Arquitectura de la Aplicación

La aplicación desarrollada sigue una arquitectura de tres capas orientada a servicios, compuesta por el backend (la base de datos) y el frontend.

1.1 Backend (API REST)

El backend fue implementado utilizando FastAPI en Python. Su función principal es exponer endpoints REST para el consumo de datos analíticos y realizar procesamiento de información mediante consultas agregadas en MongoDB.

Principales características:

- Framework: FastAPI

- Lenguaje: Python
- Despliegue: Render
- Función: procesamiento de datos y exposición de servicios REST

Endpoints implementados:

- /analytics/top-hoteles: retorna el Top 10 de hoteles por calificación promedio
- /analytics/evolucion/{id_hotel}: retorna la evolución mensual de un hotel
- /analytics/ciudad: retorna métricas agregadas por conjunto de hoteles

1.2 Base de Datos

Se utiliza MongoDB como gestor de base de datos NoSQL.

Colecciones principales:

- reseñas: contiene la información principal de las reseñas
- votos: almacena los votos de utilidad
- respuestas: almacena respuestas administrativas

Características relevantes:

- Uso de pipelines de agregación (\$match, \$group, \$sort)
- Manejo de datos heterogéneos (campos de fecha en formato Date y String)
- Conversión dinámica de datos para compatibilidad en consultas analíticas

1.3 Frontend

El frontend fue desarrollado utilizando Oracle APEX, complementado con JavaScript y la librería Chart.js para visualización de datos.

Funciones principales:

- Consumo de APIs mediante fetch
- Renderización de gráficos dinámicos
- Interacción con el usuario mediante filtros (fechas, hotel, año)

Link de la api de render: <https://dann-alpes-api.onrender.com>

Link del repo con main y requirements: [IsbellAp/dann-alpes-api](https://github.com/IsbellAp/dann-alpes-api)

2. Páginas de APEX

Usuarios creados para la aplicación:

Usuario: CLIENTE

Contraseña: CLIENTE

Usuario: ADMIN

Contraseña: ADMIN

2.1 RF1 - Crear Reseña

Descripción: Esta página permite a un cliente autenticado crear una nueva reseña sobre una de sus reservas completadas. El sistema valida que el usuario tenga una reserva en estado "Completada" y que no haya reseñado esa estada previamente.

Componentes:

- Ítems:
 - P2_ID_RESERVA (Select List poblada vía SQL con reservas Completadas)
 - P2_CALIFICACION (Select List con valores 1 a 5 estrellas)
 - P2_TEXTO (Textarea para el comentario)
 - P2_ID_HOTEL (Hidden, llenado automáticamente)
 - P2_DOCUMENTO_CLIENTE (Hidden, llenado automáticamente)
- Botón: BTN_CREAR_RESENA
- Proceso Ajax: GET_DATOS_RESERVA (PL/SQL que retorna id_hotel y documento_cliente al seleccionar la reserva)

Funcionamiento:

1. El usuario selecciona una reserva del dropdown (solo aparecen las Completadas)
2. Mediante un Dynamic Action on Change, se invoca el proceso Ajax que consulta Oracle para obtener id_hotel y documento_cliente asociados a la reserva
3. El usuario completa calificación y texto
4. Al presionar el botón, JavaScript ejecuta fetch POST al endpoint /resenas enviando el JSON con todos los campos
5. La API valida en MongoDB que no exista una reseña previa publicada para esa reserva, y si pasa, inserta el documento
6. El usuario es redirigido a "Mis Reseñas" con mensaje de confirmación

2.2 RF2 - Editar Reseña

Descripción: Permite al cliente modificar el texto y calificación de una reseña previamente creada por él mismo.

Componentes:

- Ítem: P3_ID_RESENA (Hidden, recibido vía localStorage desde "Mis Reseñas")
- Región Static Content con contenedor #editar-resena-container (renderizado dinámicamente vía JavaScript)

Funcionamiento:

1. El usuario accede a esta página desde el botón "Editar" en "Mis Reseñas" (página 7)
2. Mediante localStorage se recibe el ID de la reseña a editar
3. JavaScript invoca GET /resenas/{id} para traer los datos actuales

4. Se renderiza un formulario pre-cargado con calificación y texto
5. Al guardar, se ejecuta fetch PUT al endpoint /resenas/{id} con los nuevos valores
6. MongoDB actualiza los campos puntuacion_estrellas y descripcion

2.3 RF3 - Eliminar Reseña (Cliente)

Descripción: Permite al cliente eliminar una de sus reseñas. La reseña no se elimina físicamente: cambia su estado a "ELIMINADA" y registra quién la eliminó.

Componentes:

- Ítem: P4_ID_RESENA (Hidden)
- Región dinámica #eliminar-resena-container

Funcionamiento:

1. Se accede desde el botón "Eliminar" en "Mis Reseñas"
2. JavaScript trae los datos de la reseña vía GET /resenas/{id} y los muestra como confirmación
3. Al confirmar, se invoca DELETE /resenas/{id} enviando el usuario (APP_USER de APEX)
4. La API actualiza el documento con estado: "ELIMINADA", eliminada_por: usuario y motivo_eliminacion
5. Redirige a "Mis Reseñas"

2.4 RF4 – Reseñas del Hotel

Descripción: Página pública (no requiere autenticación) que permite a cualquier usuario consultar las reseñas publicadas de un hotel específico, con opciones de ordenamiento por fecha o utilidad.

Componentes:

- Ítems:
 - P5_ID_HOTEL (Select List poblada con todos los hoteles desde Oracle)
 - P5_ORDEN (Select List con valores "Fecha" y "Utilidad")
- Botón: BTN_BUSCAR

Funcionamiento:

1. El usuario selecciona un hotel y un criterio de orden
2. Al presionar Buscar, JavaScript ejecuta GET /resenas/hotel/{id_hotel}
3. La API consulta MongoDB, filtra por estado: "publicada", cuenta votos desde la colección votos y busca respuestas desde la colección respuestas
4. Se renderizan dinámicamente las reseñas mostrando: estrellas, texto, fecha, conteo de votos útiles, respuesta del hotel (si existe) y botón para marcar como útil

Justificación técnica: Se renderizan las reseñas con HTML dinámico generado por JavaScript en lugar de usar un Classic Report de APEX, porque los datos vienen de MongoDB (no de Oracle) y porque se requiere un diseño visualmente atractivo con estrellas, badges de estado y formato de tarjetas.

2.5 RF5 – Marcar Reseña como Útil

Descripción: Permite a un usuario autenticado registrar un voto de utilidad sobre una reseña específica. Cada voto se almacena en la colección votos de MongoDB.

Componentes:

- Ítem: P6_ID_RESENA (Hidden)
- Región dinámica #util-resena-container

Funcionamiento:

1. Se accede desde el botón "Útil" en la página de Reseñas del Hotel
2. JavaScript trae los datos de la reseña vía GET /resenas/{id}
3. Se muestra una vista previa con botón de confirmación
4. Al confirmar, se invoca POST /resenas/{id}/util con el documento_cliente
5. La API crea un nuevo documento en la colección votos con campos: id_voto_de_utilidad, documento_cliente, id_reseña, pulgar_arriba: true y fecha_voto
6. Redirige a Reseñas del Hotel

Justificación de diseño: Los votos se almacenan en una colección separada (no como contador en la reseña) para mantener trazabilidad, permitir análisis posterior y respetar la integridad referencial del esquema de MongoDB. El conteo se calcula dinámicamente al traer las reseñas.

2.6 RF6 – Mis Reseñas

Descripción: Permite al cliente autenticado consultar el historial completo de sus reseñas, incluyendo las eliminadas, con opciones de ordenamiento por fecha o por hotel.

Componentes:

- Ítem: P7_ORDEN (Select List "Fecha" / "Hotel")
- Botón: BTN_BUSCAR_RESENAS
- Región dinámica #mis-resenas-container

Funcionamiento:

1. El usuario selecciona un orden y presiona Buscar
2. JavaScript ejecuta GET /resenas/cliente/{documento_cliente}
3. La API consulta MongoDB usando un filtro \$or que acepta tanto int como Int64 (manejando la heterogeneidad de tipos entre los datos poblados y los recién creados)

4. Se renderizan las reseñas mostrando: estado (badge verde/rojo), estrellas, texto, fecha, votos, indicador de respuesta del hotel y botones de acción
5. Los botones "Editar" y "Eliminar" solo aparecen si la reseña está en estado "publicada"

Justificación técnica: Se mantiene la trazabilidad mostrando también las reseñas eliminadas (con badge rojo) para que el cliente vea su historial completo, pero se ocultan las acciones de modificación sobre reseñas que ya están eliminadas.

2.7 RF7 – Responder Reseña (Administrador)

Descripción: Permite a un administrador agregar o editar una respuesta oficial a una reseña de su hotel.

Componentes:

- Ítem: P9_ID_RESENA (Hidden)
- Región dinámica #responder-resena-container

Funcionamiento:

1. Se accede desde el Panel Admin (página 8) al hacer clic en "Responder"
2. JavaScript busca la reseña en los hoteles disponibles
3. Se muestra la reseña original y un textarea pre-cargado con la respuesta actual (si existe)
4. Al enviar, se invoca PUT /resenas/{id}/respuesta con el texto
5. La API verifica si ya existe una respuesta para esa reseña:
 - a. Si existe: actualiza el documento en respuestas
 - b. Si no existe: crea un nuevo documento con id_respuesta_administrativa, id_reseña, documento_administrador, descripcion, fecha y hora
6. Redirige al Panel Admin

Justificación de diseño: Las respuestas se almacenan en una colección separada (no como campo embebido en la reseña) para mantener un esquema normalizado, permitir votos de utilidad sobre las respuestas mismas y respetar el modelo conceptual definido en la entrega.

2.8 RF8 – Eliminar Reseña (Administrador)

Descripción: Permite al administrador eliminar reseñas que considere violan las políticas de convivencia digital, registrando el motivo y el usuario que realizó la acción.

Componentes:

- Ítem: P10_ID_RESENA (Hidden)
- Región dinámica #eliminar-admin-container

Funcionamiento:

1. Se accede desde el Panel Admin al hacer clic en "Eliminar"
2. JavaScript trae los datos de la reseña y los muestra
3. El administrador debe escribir un motivo de eliminación (obligatorio)
4. Al confirmar, se invoca DELETE /resenas/{id}/admin enviando motivo y usuario (APP_USER)
5. La API actualiza el documento con estado: "ELIMINADA", eliminada_por: usuario_admin y motivo_eliminacion: motivo
6. Redirige al Panel Admin

Justificación de diseño: A diferencia del RF3 (eliminación por cliente), el administrador debe especificar obligatoriamente un motivo, lo cual queda registrado para auditoría y eventuales reclamos del cliente cuya reseña fue eliminada.

2.9 RF9 – Destacar Reseña

Descripción: Permite al administrador marcar una reseña como “destacada” para que aparezca fijada al tope de la sección de reseñas del hotel. Solo puede haber una reseña destacada por hotel a la vez.

Componentes:

- Ítem: P11_ID_RESENA (Hidden)
- Región dinámica #destacar-resena-container

Funcionamiento:

1. Se accede desde el Panel Admin al hacer clic en “Destacar”
2. JavaScript trae los datos de la reseña y muestra si ya está destacada o no
3. Al confirmar, se invoca POST /resenas/{id}/destacar enviando el id_hotel
4. La API ejecuta dos operaciones atómicas:
 - a. updateMany: marca como destacada: false todas las reseñas del hotel
 - b. updateOne: marca como destacada: true la reseña seleccionada
5. Esto garantiza la regla de negocio "máximo una reseña destacada por hotel"

Justificación técnica: El orden de las dos operaciones es crítico: primero se desmarcan todas las del hotel, y luego se marca la nueva. Si se invirtiera el orden, podríamos quedarnos sin reseña destacada momentáneamente o desmarcarnos a nosotros mismos.

2.10 Panel de Administración de Reseñas

Descripción: Página adicional (no incluida en los RF originales pero necesaria para la operación) que permite al administrador visualizar todas las reseñas de uno de sus hoteles y acceder a las acciones de Responder, Eliminar y Destacar.

Componentes:

- Ítem: P8_ID_HOTEL (Select List poblada con los hoteles administrados por el usuario autenticado)
- Botón: BTN_BUSCAR_ADMIN
- Región dinámica #admin-resenas-container

Funcionamiento:

1. El administrador selecciona uno de sus hoteles
2. JavaScript ejecuta GET /resenas/hotel/{id_hotel}
3. Se renderizan todas las reseñas (incluidas las destacadas con borde amarillo amarillo)
4. Cada reseña tiene tres botones: Responder, Destacar y Eliminar
5. Cada acción guarda el ID en localStorage y redirige a la página correspondiente

Justificación de existencia: Aunque no es un RF explícito, esta página actúa como una conexión desde donde se ejecutan los RF7, RF8 y RF9, mejorando significativamente la experiencia del administrador al no requerir que conozca el ID de cada reseña.

2.11. RFC1 – Top de Hoteles por Calificación

Descripción:

Esta página permite visualizar el Top 10 de hoteles con base en su calificación promedio dentro de un rango de fechas seleccionado por el usuario.

Componentes:

- Ítems:
 - P12_FECHA_INICIO
 - P12_FECHA_FIN
- Botón:
 - Consultar Top 10
- Región gráfica:

<canvas id="rfc1Chart"></canvas>

Funcionamiento:

1. El usuario selecciona un rango de fechas.
2. Se realiza una solicitud HTTP al endpoint /analytics/top-hoteles.
3. La API retorna los datos agregados.
4. Se renderiza un gráfico combinado:
 - a. Barras: calificación promedio
 - b. Línea: número de reseñas

Resultado esperado:

- Se muestran los 10 hoteles con mayor calificación promedio
- Los datos se encuentran ordenados de mayor a menor

- Se visualizan simultáneamente promedio y volumen de reseñas

2.12. RFC2 – Evolución Mensual de un Hotel

Descripción:

Esta página permite visualizar el comportamiento mensual de las calificaciones de un hotel específico durante un año determinado.

Componentes:

- Ítems:
 - P13_HOTEL_ID
 - P13_ANIO
- Región gráfica:

<canvas id="rfc2Chart"></canvas>

Funcionamiento:

1. El usuario selecciona un hotel y un año.
2. Se realiza una solicitud al endpoint /analytics/evolucion/{id_hotel}.
3. La API filtra y agrupa la información por mes.
4. Se renderiza un gráfico combinado:
 - a. Línea: calificación promedio mensual
 - b. Barras: cantidad de reseñas

Procesamiento en backend:

El pipeline de agregación realiza:

- Conversión de fechas mediante \$dateToString
- Filtrado por hotel y año
- Agrupación por mes (YYYY-MM)
- Cálculo de promedio y conteo de reseñas
- Orden cronológico de resultados

Resultado esperado:

- Eje X: meses del año
- Eje Y izquierdo: calificación (0 a 5)
- Eje Y derecho: número de reseñas

2.14. RFC3 – Perfil Comparativo de Hoteles por Ciudad

Descripción:

Esta página permite visualizar un análisis comparativo de todos los hoteles asociados a una ciudad seleccionada. El sistema calcula métricas agregadas a partir de las reseñas, incluyendo la calificación promedio, el total de reseñas, el porcentaje de reseñas destacadas y el porcentaje de reseñas con respuesta administrativa.

Además, permite identificar aquellos hoteles cuyo desempeño se encuentra por debajo del promedio general de la ciudad, apoyando la toma de decisiones del administrador.

Componentes:

Ítems:

- **P14_ID_CIUADAD**
Lista desplegable (Select List) que permite seleccionar la ciudad a analizar.
Se llena con datos provenientes de Oracle (tabla de ciudades).
- **P14_HOTELES_CSV**
Ítem oculto que almacena los IDs de los hoteles asociados a la ciudad seleccionada en formato CSV.

Botón:

- **Comparar Hoteles**
Ejecuta el proceso de consulta y visualización de los datos.

Regiones:

- **rfc3-loading**
Indicador de carga que se muestra mientras se procesa la consulta.
- **rfc3-kpis**
Región que contiene indicadores agregados (KPIs), como:
 - total de hoteles analizados
 - promedio de la ciudad
 - total de reseñas
 - cantidad de hoteles por debajo del promedio

- rfc3-perfilComparativo
Región dinámica (HTML) donde se renderiza la tabla comparativa de hoteles.

Funcionamiento:

1. El usuario selecciona una ciudad desde el ítem P14_ID_CIUADAD.
2. Al presionar el botón **Comparar Hoteles**, se ejecuta un proceso AJAX en Oracle APEX (GET_HOTELES_BY_CIUADAD) que consulta la base de datos relacional y retorna los IDs de los hoteles asociados a la ciudad seleccionada.
3. Estos IDs se almacenan en el ítem oculto P14_HOTELES_CSV.
4. Se construye una solicitud HTTP hacia el endpoint del backend:

/analytics/ciudad?hoteles=<lista_de_ids>

1. El backend (FastAPI) ejecuta un pipeline de agregación en MongoDB que:
 - a. filtra reseñas por hotel y estado
 - b. realiza un \$lookup con respuestas administrativas
 - c. calcula promedios, totales y porcentajes
 - d. calcula el promedio general de la ciudad
 - e. determina cuáles hoteles están por debajo del promedio
2. La API retorna un arreglo JSON con los indicadores por hotel.
3. En el frontend (JavaScript dentro de APEX):
 - a. se oculta el loader
 - b. se procesan los datos recibidos
 - c. se calculan indicadores adicionales (KPIs globales)
4. Se renderiza dinámicamente una tabla HTML que incluye:
 - a. ID del hotel
 - b. calificación promedio
 - c. total de reseñas
 - d. porcentaje de reseñas con respuesta
 - e. porcentaje de reseñas destacadas

- f. estado del hotel respecto al promedio
- 5. Las filas de la tabla se resaltan visualmente:
 - a. hoteles por debajo del promedio → color diferenciado
 - b. hoteles en promedio o superior → estilo neutro
- 6. En caso de que no existan reseñas para los hoteles de la ciudad:
 - la API retorna un arreglo vacío
 - el sistema muestra el mensaje:
“Sin reseñas para esta ciudad”

Resultado esperado:

- Se muestra una tabla comparativa con los hoteles de la ciudad seleccionada
- Cada hotel presenta los siguientes indicadores:
 - calificación promedio
 - total de reseñas
 - porcentaje de reseñas destacadas
 - porcentaje de reseñas con respuesta administrativa
- Se calcula y muestra el promedio general de la ciudad
- Se identifican claramente los hoteles que se encuentran por debajo del promedio
- La información se presenta de forma ordenada (de mayor a menor calificación promedio)
- El sistema maneja correctamente escenarios sin datos mostrando mensajes informativos y evitando errores en la interfaz

5. Pruebas de Requerimientos Funcionales

Los requerimientos funcionales de consulta (RFC1, RFC2 y RFC3) están diseñados para el rol de administrador del sistema, ya que permiten el análisis agregado de las reseñas, el monitoreo del desempeño de los hoteles y la identificación de oportunidades de mejora. Estos análisis no son visibles para clientes finales, debido a que contienen información consolidada y estratégica que es utilizada para la gestión interna del sistema.

5.1 RFC1 – Top de Hoteles

Prueba realizada:

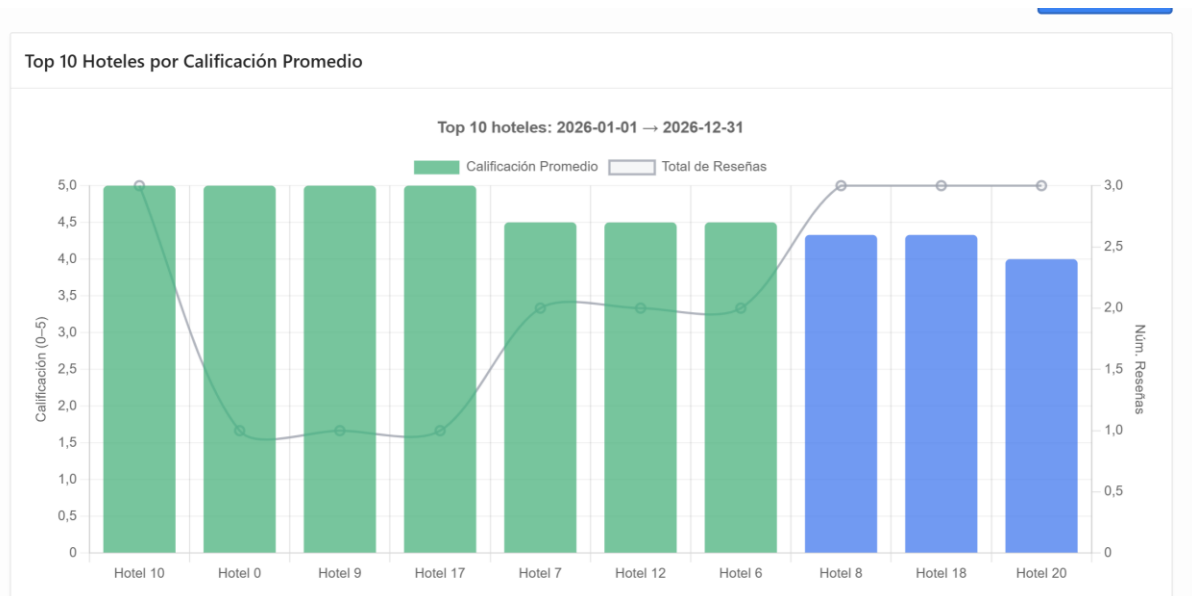
- Se ingresó un rango de fechas válido
- Se ejecutó la consulta desde la interfaz APEX

Filtros de Búsqueda

Fecha Inicio 2026-01-01	Fecha Fin 2026-12-31
----------------------------	-------------------------

Consultar Top 10

Resultado:



- Se visualizaron correctamente los hoteles
- Se observó la relación entre calificación promedio y número de reseñas

Validación:

- La API respondió correctamente
- El gráfico se renderizó sin errores
- Los datos mostrados son coherentes

5.1 RFC1 – Top de Hoteles (caso sin datos suficientes)

Prueba realizada:

Se ingresó un rango de fechas válido desde la interfaz APEX (por ejemplo, un rango donde no existen reseñas registradas en la base de datos)

Se ejecutó la consulta presionando el botón “Consultar Top 10”

RFC1 - Top Hoteles por Calificación

Filtros de Búsqueda

Fecha Inicio 2026-01-01	Fecha Fin 2026-01-02
----------------------------	-------------------------

Consultar Top 10

Resultado:

RFC1 - Top Hoteles por Calificación

Filtros de Búsqueda

Fecha Inicio 2026-01-01	Fecha Fin 2026-01-02
----------------------------	-------------------------

Consultar Top 10

Top 10 Hoteles por Calificación Promedio

No se encontraron reseñas en el período seleccionado.

No se visualizaron datos en el gráfico de resultados debido a la ausencia de reseñas en el rango de fechas seleccionado

El sistema mostró el mensaje informativo:

“No se encontraron reseñas en el período seleccionado”

Se evidenció que el sistema maneja correctamente escenarios donde no existen datos para realizar el cálculo del Top 10 de hoteles

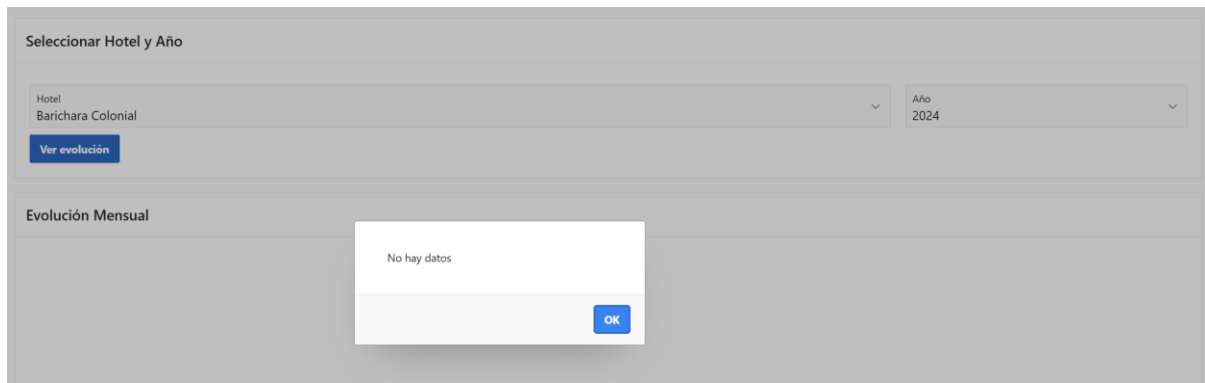
Validación:

- La API respondió correctamente retornando un conjunto vacío de datos
- El pipeline de agregación se ejecutó sin errores, confirmando que el filtrado por fechas funciona adecuadamente
- Se verificó que el sistema evita mostrar información incorrecta o incompleta cuando no hay registros disponibles
- La interfaz de APEX gestionó correctamente este caso mostrando un mensaje claro al usuario, asegurando una adecuada experiencia de uso
- Se confirmó que la ausencia de resultados corresponde a la falta de reseñas en el rango de fechas seleccionado y no a errores en la implementación

5.2 RFC2 – Evolución Mensual

Prueba realizada:

- Se seleccionó un hotel específico
- Se definió un año de consulta



No se visualizaron datos en el gráfico de evolución mensual debido a la ausencia de reseñas registradas para el hotel en el año seleccionado

El sistema mostró un mensaje emergente informativo:

“No hay datos”

Se evidenció que el sistema maneja correctamente escenarios en los que no existen registros para el análisis temporal

Validación:

- La API respondió correctamente retornando un conjunto vacío de datos para el hotel y año seleccionado
- El pipeline de agregación se ejecutó sin errores, confirmando que el filtrado por hotel y año funciona adecuadamente
- Se verificó que el sistema no intenta renderizar gráficos con datos inexistentes, evitando errores de visualización
- La interfaz de APEX gestionó correctamente este caso mediante un mensaje claro al usuario, garantizando una adecuada experiencia de uso
- Se confirmó que la ausencia de resultados corresponde a la falta de reseñas en la base de datos para el hotel en el período seleccionado y no a fallos en la implementación

5.3 RFC3 – Perfil comparativo de hoteles por ciudad

Prueba realizada:

Se seleccionó una ciudad desde la interfaz APEX

Se ejecutó la consulta que obtiene los hoteles asociados desde Oracle

Se realizó la llamada al endpoint /analytics/ciudad con los IDs de hoteles

RFC3 - Perfil Comparativo por Ciudad

Seleccionar Ciudad

Ciudad
Medellín

Comparar Hoteles

Resultado:

RFC3 - Perfil Comparativo por Ciudad

Seleccionar Ciudad

Ciudad
Medellín

Comparar Hoteles

Perfil Comparativo

HOTELES EN CIUDAD 1	PROMEDIO CIUDAD 4.00 ★	TOTAL RESEÑAS 1	BAJO PROMEDIO 0 hotel(es)
-------------------------------	----------------------------------	---------------------------	-------------------------------------

Hotel	Promedio	Reseñas	% Resp	% Dest	Estado
Hotel 2	4.00	1	0.0%	0.0%	En promedio

Se visualizaron correctamente los hoteles pertenecientes a la ciudad seleccionada

Se observó la comparación entre:

- Calificación promedio por hotel
- Total de reseñas
- Porcentaje de reseñas destacadas
- Porcentaje de reseñas con respuesta administrativa

Se evidenció la identificación de hoteles por debajo del promedio general de la ciudad

*en este caso es importante la cantidad de tuplas en la db

Validación:

La API respondió correctamente con un conjunto de datos estructurado por hotel

El pipeline de agregación se ejecutó sin errores, incluyendo el uso de \$lookup para obtener respuestas administrativas

Los porcentajes fueron calculados correctamente en el backend mediante operaciones de agregación

Se verificó que el promedio de la ciudad se calculó correctamente y permitió comparar el desempeño relativo de cada hotel

Los datos mostrados son consistentes con la información almacenada en las colecciones Reseñas y Respuestas_Administrativas

5.3 RFC3 – Perfil comparativo de hoteles por ciudad (caso sin datos)

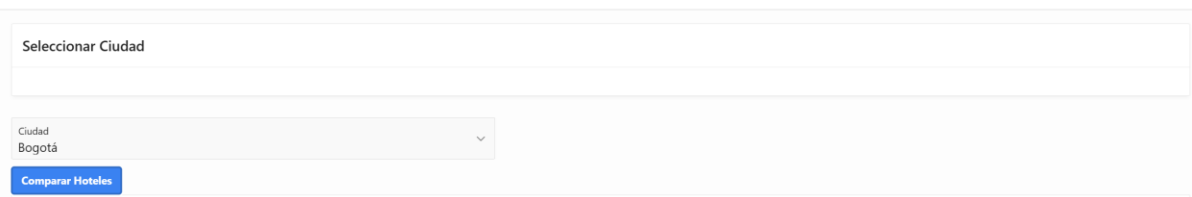
Prueba realizada:

Se seleccionó una ciudad desde la interfaz APEX (en este caso, Bogotá)

Se ejecutó la consulta que obtiene los hoteles asociados a la ciudad desde Oracle

Se realizó la llamada al endpoint /analytics/ciudad con los IDs de hoteles obtenidos

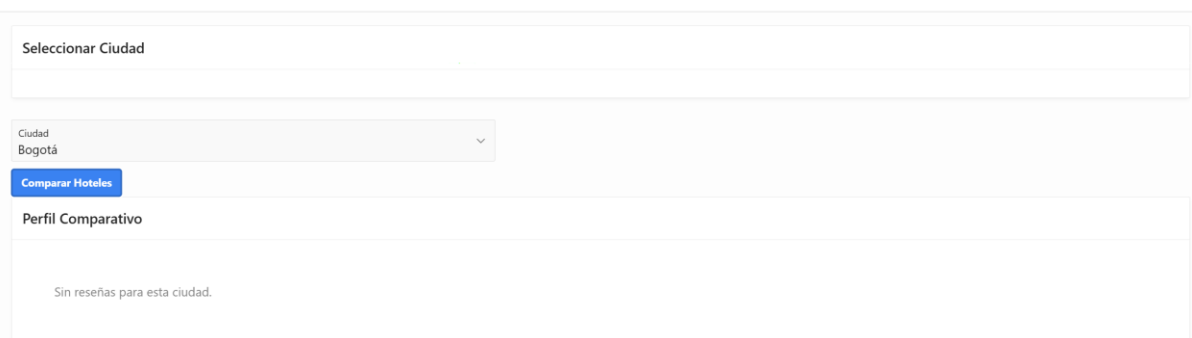
RFC3 - Perfil Comparativo por Ciudad



The screenshot shows a web interface titled "RFC3 - Perfil Comparativo por Ciudad". It features a text input field labeled "Seleccionar Ciudad" with a dropdown arrow. Below it, a dropdown menu is open, showing "Ciudad" and "Bogotá". A blue button labeled "Comparar Hoteles" is positioned below the dropdown.

Resultado:

RFC3 - Perfil Comparativo por Ciudad



The screenshot shows the same web interface as before, but now with a message displayed below the "Comparar Hoteles" button. The message reads "Sin reseñas para esta ciudad." (No reviews for this city).

No se visualizaron datos en la tabla comparativa debido a la ausencia de reseñas asociadas a los hoteles de la ciudad seleccionada

El sistema mostró el mensaje informativo:

“Sin reseñas para esta ciudad”

Se evidenció que el sistema maneja correctamente casos en los que no existen datos para realizar el análisis

Validación:

- La API respondió correctamente, retornando un arreglo vacío sin generar errores
- El pipeline de agregación se ejecutó sin fallos, confirmando que el filtrado por ciudad y estado funciona adecuadamente
- Se verificó que el sistema identifica correctamente la ausencia de datos y evita mostrar información incorrecta o incompleta
- La interfaz de APEX gestionó correctamente este caso mostrando un mensaje claro al usuario, garantizando una adecuada experiencia de uso
- Se confirmó que la ausencia de resultados corresponde a la falta de reseñas en la base de datos para los hoteles de la ciudad seleccionada, y no a errores en la implementación

5.1 RF1 – Crear Reseña

Prueba realizada:

- Se autenticó un usuario con rol CLIENTE
- Se seleccionó una reserva con estado "Completada" del dropdown
- Se asignó una calificación de 5 estrellas
- Se escribió un texto de prueba en el textarea
- Se presionó el botón "Crear Reseña"

Resultado:

- El sistema mostró el mensaje "¡Reseña creada exitosamente!"
- El usuario fue redirigido automáticamente a la página "Mis Reseñas"
- La nueva reseña apareció en el historial del cliente

iacademy2.oracle.com dice

cliente

Crear Reseña

¡Reseña creada exitosamente!

Aceptar

Crear Reseña

Selecciona tu reserva
Reserva #5 - Hotel: Brisa del Rodadero (15/05/2026 al 20/05/2026)

Calificación
★★★★★ 5 - Excelente

Escribe tu reseña
Muy bueno todo

Crear Reseña

Mis Reseñas

Ordenar por
Fecha

Buscar

Hotel #5

★★★★★ 1/5

Que mierda de experiencia, el servicio al cliente es una basura, muy indignado.

2026-05-20 0 útil(es)

eliminada

Hotel #4

★★★★★ 1/5

Qué experiencia tan horrible, el personal fue grosero y el servicio una porquería.

2026-05-19 0 útil(es) Con respuesta del hotel

eliminada

Hotel #5

★★★★★ 5/5

Muy bueno todo

2026-05-26 0 útil(es)

Editar Eliminar

publicada

id: ObjectId('6a162c8a913fb9a5846d853a')

id_reseña: 91602683

id_hotel: 5

documento_cliente: 6224269167

id_reserva: 5

puntuacion_estrellas: 5

descripcion: "Muy bueno todo"

fecha: "2026-05-26"

hora: "23:28"

estado: "publicada"

destacada: false

motivo Eliminacion: null

eliminada_por: null

Validación:

- La API respondió con código 200 y el `_id` del nuevo documento
- En MongoDB Compass se verificó la creación del documento en la colección `resenas` con todos los campos correctos: `id_reseña`, `id_hotel`, `documento_cliente`, `id_reserva`, `puntuacion_estrellas`, `descripcion`, `fecha`, `hora`, `estado`: "publicada", `destacada`: false

- Se confirmó la restricción de unicidad: al intentar crear una segunda reseña para la misma reserva, la API retornó error 400 con mensaje "Ya existe una reseña activa para esta reserva"

5.2 RF2 – Editar Reseña

Prueba realizada:

- Desde "Mis Reseñas" se hizo clic en el botón "Editar" de una reseña previamente creada
- Se modificó la calificación de 5 a 3 estrellas
- Se cambió el texto del comentario
- Se presionó "Guardar Cambios"

Resultado:

- El sistema mostró el mensaje "¡Reseña actualizada exitosamente!"
- El usuario fue redirigido a "Mis Reseñas" donde se visualizó la reseña con los nuevos valores

Hotel #5

★★★★★ 5/5

Muy bueno todo



2026-05-26



0 útil(es)



Editar



Eliminar

iacademy2.oracle.com dice

¡Reseña actualizada exitosamente!

Aceptar

cliente

Editar Reseña

Hotel #5

Calificación:

★★★ (3/5)

Texto:

Muy bueno todo, pero el servicio es un poco regular

Guardar Cambios

cliente

Mis Reseñas

Buscar

Hotel #5

eliminado

★★★★★ 1/5

Que mierda de experiencia, el servicio al cliente es una basura, muy indignado.

2026-05-20 0 útil(es)

Hotel #4

eliminado

★★★★★ 1/5

Qué experiencia tan horrible, el personal fue grosero y el servicio una porquería.

2026-05-19 0 útil(es) Con respuesta del hotel

Hotel #5

publicada

★★★★★ 3/5

Muy bueno todo, pero el servicio es un poco regular

2026-05-26 0 útil(es)

Editar Eliminar

Release 1.0

App 16024 Page 7 Session Debug Quick Edit Customize

_id: ObjectId('6a162c8a913fb9a5846d853a')

id_resena: 91602683

id_hotel: 5

documento_cliente: 6224269167

id_reserva: 5

puntuacion_estrellas: 3

descripcion: "Muy bueno todo, pero el servicio es un poco regular"

fecha: "2026-05-26"

hora: "23:28"

estado: "publicada"

destacada: false

motivo_eliminacion: null

eliminada_por: null

6a162c8a913fb9a5846d853a

Validación:

- La API respondió correctamente al PUT /resenas/{id}

- En MongoDB Compass se verificó que los campos puntuacion_estrellas y descripcion se actualizaron
- Los demás campos (id_hotel, documento_cliente, fecha) permanecieron sin cambios, garantizando la integridad

5.3 RF3 – Eliminar Reseña (Cliente)

Prueba realizada:

- Desde "Mis Reseñas" se hizo clic en "Eliminar" sobre una reseña publicada
- Se confirmó la acción en el diálogo de confirmación

Resultado:

- El sistema mostró "¡Reseña eliminada exitosamente!"
- El usuario fue redirigido a "Mis Reseñas"
- La reseña apareció con badge rojo "ELIMINADA" y sin los botones de edición

Validación:

- En MongoDB se verificó que el documento no fue borrado físicamente
- El campo estado cambió a "ELIMINADA"
- Se registró correctamente eliminada_por: <usuario> y motivo_eliminacion: "Eliminada por el cliente"
- La reseña dejó de aparecer en las consultas públicas (RF4) pero sigue en el historial del cliente (RF6)

Eliminar Reseña

Eliminar reseña

¿Estás seguro de eliminar esta reseña?

Hotel #5

★★★★☆ 3/5

Muy bueno todo, pero el servicio es un poco regular

2026-05-26

 **Sí, Eliminar**

Cancelar

iacademy2.oracle.com dice

 cliente

Eliminar Reseña

¿Estás seguro de eliminar esta reseña?

Aceptar

Cancelar

Eliminar reseña

¿Estás seguro de eliminar esta reseña?

Hotel #5

★★★★☆ 3/5

Muy bueno todo, pero el servicio es un poco regular

2026-05-26

 **Sí, Eliminar**

Cancelar

Mis Reseñas

Ordenar por
Fecha

Buscar

Hotel #5

eliminada

☆☆☆☆☆ 1/5

Que mierda de experiencia, el servicio al cliente es una basura, muy indignado.

2026-05-20 0 útil(es)

Hotel #4

eliminada

☆☆☆☆☆ 1/5

Qué experiencia tan horrible, el personal fue grosero y el servicio una porquería.

2026-05-19 0 útil(es) Con respuesta del hotel

Hotel #5

ELIMINADA

☆☆☆☆☆ 3/5

Muy bueno todo, pero el servicio es un poco regular

2026-05-26 0 útil(es)

▶

```
_id: ObjectId('6a162c8a913fb9a5846d853a')
id_reseña: 91602683
id_hotel: 5
documento_cliente: 6224269167
id_reserva: 5
puntuacion_estrellas: 3
descripcion: "Muy bueno todo, pero el servicio es un poco regular"
fecha: "2026-05-26"
hora: "23:28"
estado: "ELIMINADA"
destacada: false
motivo Eliminacion: "Eliminada por el cliente"
eliminada_por: "cliente"
```

✎

👤

🗨

🗑

5.4 RF4 – Consultar Reseñas de un Hotel

Prueba realizada:

- Se accedió a la página "Reseñas del Hotel" sin autenticación
- Se seleccionó el Hotel del dropdown
- Se eligió el orden por "Fecha"
- Se presionó "Buscar"

Resultado:

- Se visualizaron todas las reseñas publicadas del hotel ordenadas de la más reciente a la más antigua
- Cada reseña mostró: calificación en estrellas, texto, fecha, contador de votos útiles y respuesta del administrador (si existía)

Reseñas del Hotel

Filtros

Selecciona un hotel

Barichara Colonial

▼

Ordenar por

Fecha

▼

Buscar

Reseñas

★★★★★ 5/5

Una experiencia fantástica, seguramente volveré con mi familia.

2026-06-12

👍 2 útil(es)

Útil

Respuesta del hotel:

Nos alegra que haya disfrutado su experiencia familiar.

Validación:

- La API filtró correctamente por estado: "publicada" excluyendo las eliminadas
- El conteo de votos útiles se calculó dinámicamente desde la colección `votos`
- Las respuestas administrativas se trajeron desde la colección `respuestas` y se mostraron embebidas

5.5 RF5 – Marcar Reseña como Útil

Prueba realizada:

- Desde la página "Reseñas del Hotel" se hizo clic en "Útil" en una reseña
- Se confirmó la acción

Resultado:

- El sistema mostró "¡Voto registrado!"
- Al regresar a la página de reseñas, el contador de "útil(es)" aumentó en 1

Marcar Reseña Util

Marcar como Util

¿Encontraste útil esta reseña?

Hotel #17

★★★★★ 5/5

Una experiencia fantástica, seguramente volveré con mi familia.

👍 2 útil(es)

👍 Sí, marcar útil

Cancelar

Reseñas del Hotel

Filtros

Selecciona un hotel

Barichara Colonial

Ordenar por

Fecha

Buscar

Reseñas

★★★★★ 5/5

Una experiencia fantástica, seguramente volveré con mi familia.

📅 2026-06-12

👍 3 útil(es)

👍 Útil

🗨 Respuesta del hotel:

Nos alegra que haya disfrutado su experiencia familiar.

▶

```
_id: ObjectId('6a162daa913fb9a5846d853b')
id_voto_de_utilidad: 87977729
documento_cliente: 0
id_reseña: 48193
id_respuesta_administrativa: null
pulgar_arriba: true
fecha_voto: 2026-05-26T23:32:58.191+00:00
```

✎

👤

📄

🗑

87977729

Validación:

- En MongoDB Compass se verificó la creación de un nuevo documento en la colección votos con: id_voto_de_utilidad, documento_cliente, id_reseña, pulgar_arriba: true, fecha_voto
- El documento de la reseña original NO fue modificado, manteniendo la separación de colecciones
- El conteo en la interfaz refleja el total calculado dinámicamente

5.6 RF6 – Historial de Reseñas Propias

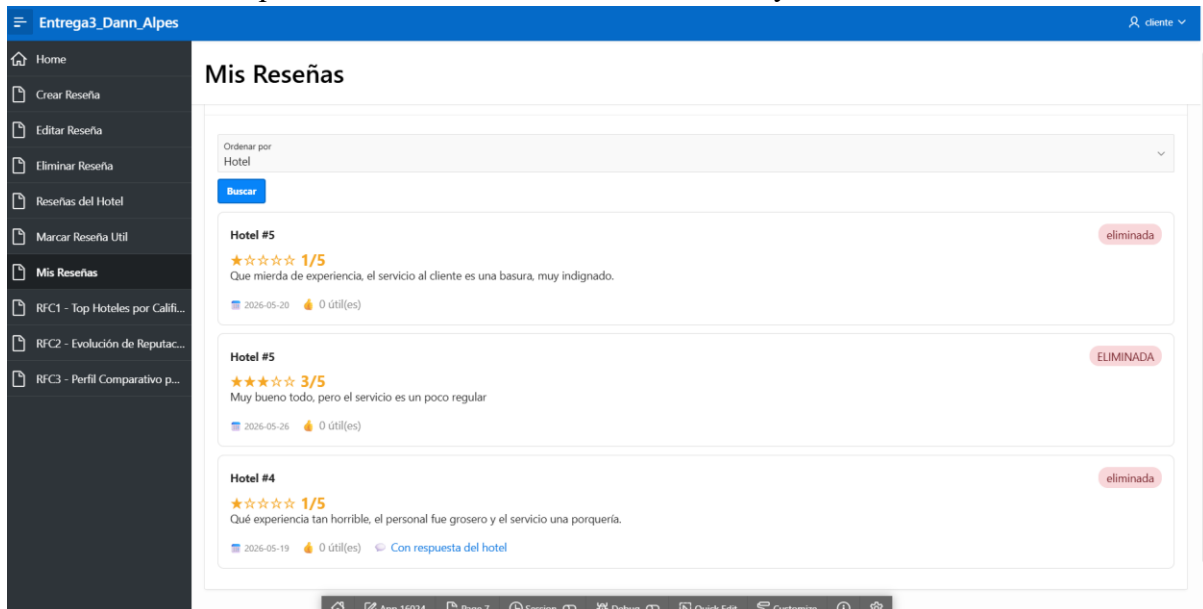
Prueba realizada:

- Autenticado como CLIENTE, se accedió a "Mis Reseñas"

- Se seleccionó el orden por "Hotel"
- Se presionó "Buscar"

Resultado:

- Se visualizaron todas las reseñas del cliente (publicadas y eliminadas)
- Las reseñas eliminadas aparecieron con badge rojo, sin botones de acción
- Las reseñas publicadas mostraron botones de Editar y Eliminar



Validación:

- La API trajo las reseñas filtrando por documento_cliente con soporte para tipos int
- El conteo de votos útiles se mostró correctamente
- El indicador "Con respuesta del hotel" apareció en las reseñas que tienen respuesta administrativa asociada

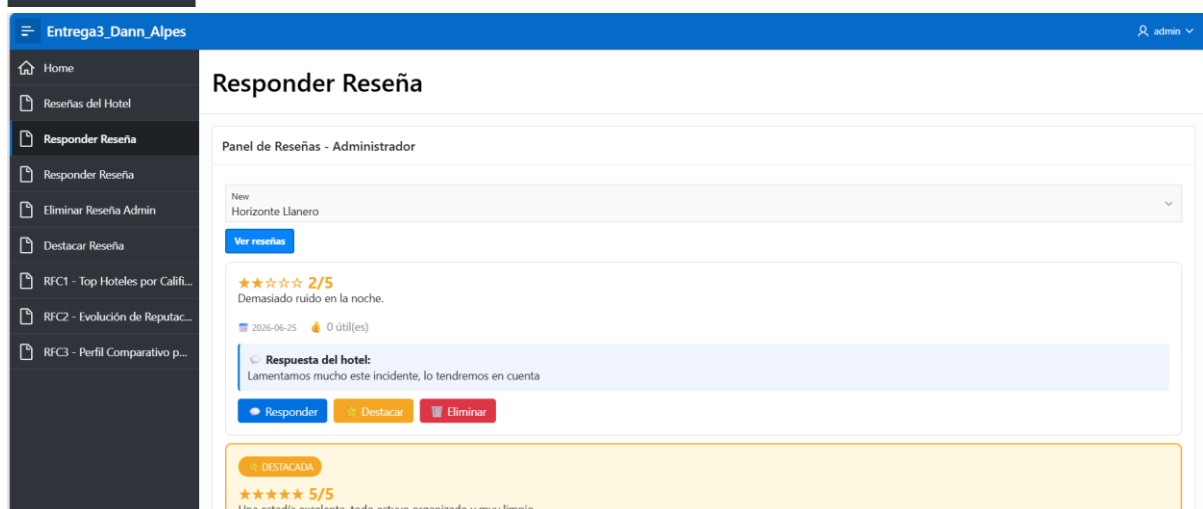
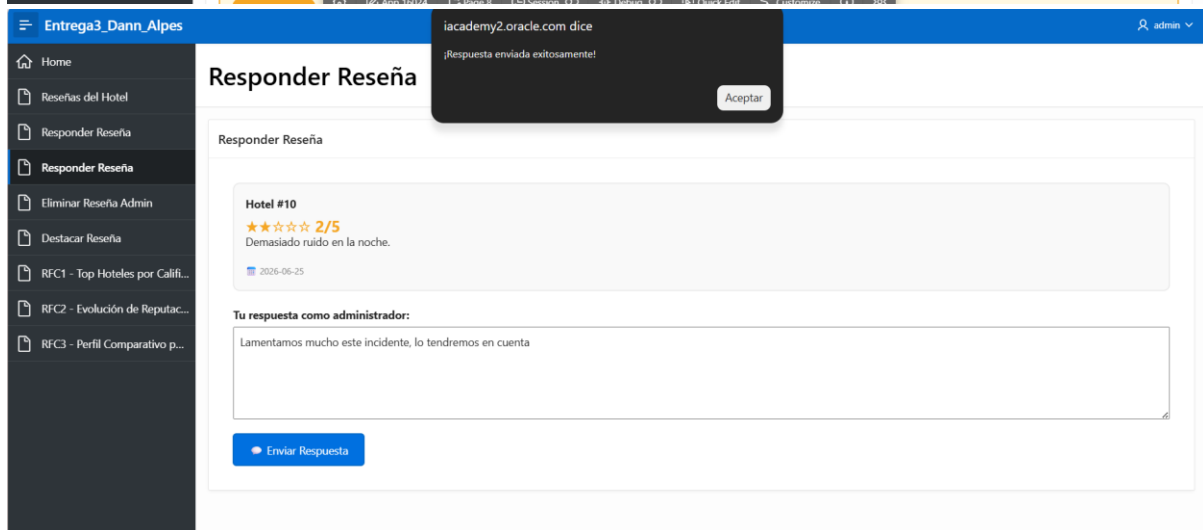
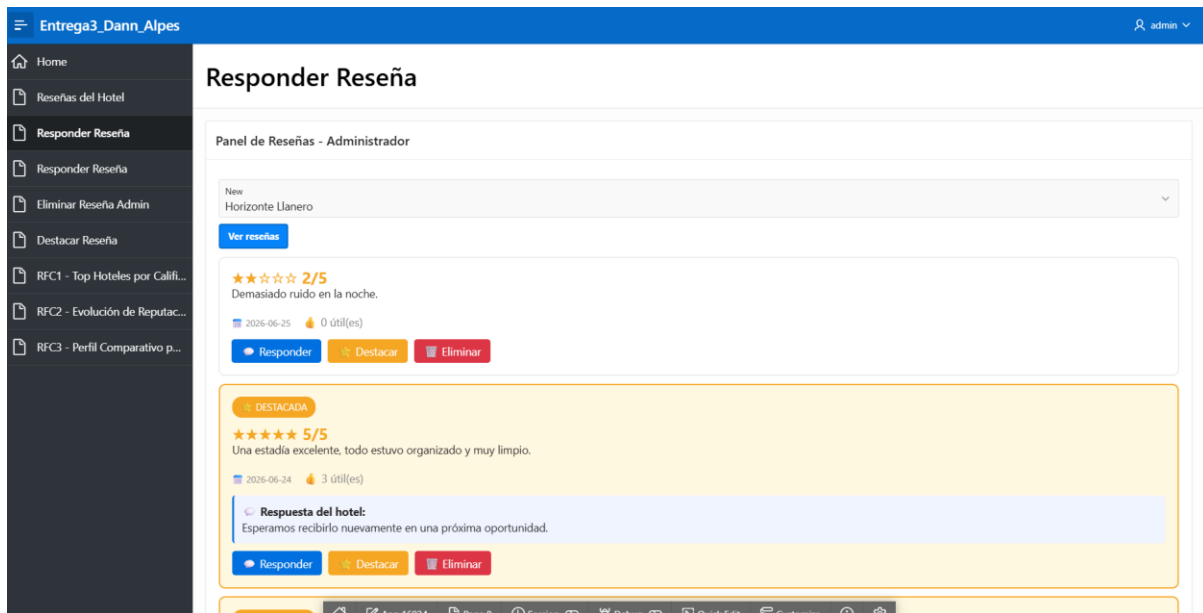
5.7 RF7 – Responder Reseña (Administrador)

Prueba realizada:

- Autenticado como ADMIN, se accedió al Responder Reseña
- Se seleccionó un hotel administrado
- Se hizo clic en "Responder" sobre una reseña
- Se escribió un texto de respuesta y se presionó "Enviar Respuesta"

Resultado:

- El sistema mostró "¡Respuesta enviada exitosamente!"
- Al regresar al Panel Admin, la respuesta apareció debajo de la reseña en color azul



Validación:

- En MongoDB se verificó la creación de un documento en la colección respuestas con todos los campos: id_respuesta_administrativa, id_reseña, documento_administrador, descripcion, fecha, hora

- Al editar una respuesta existente, se actualizó el mismo documento en lugar de crear uno nuevo
- La respuesta apareció también en la vista pública (RF4) embebida en la reseña

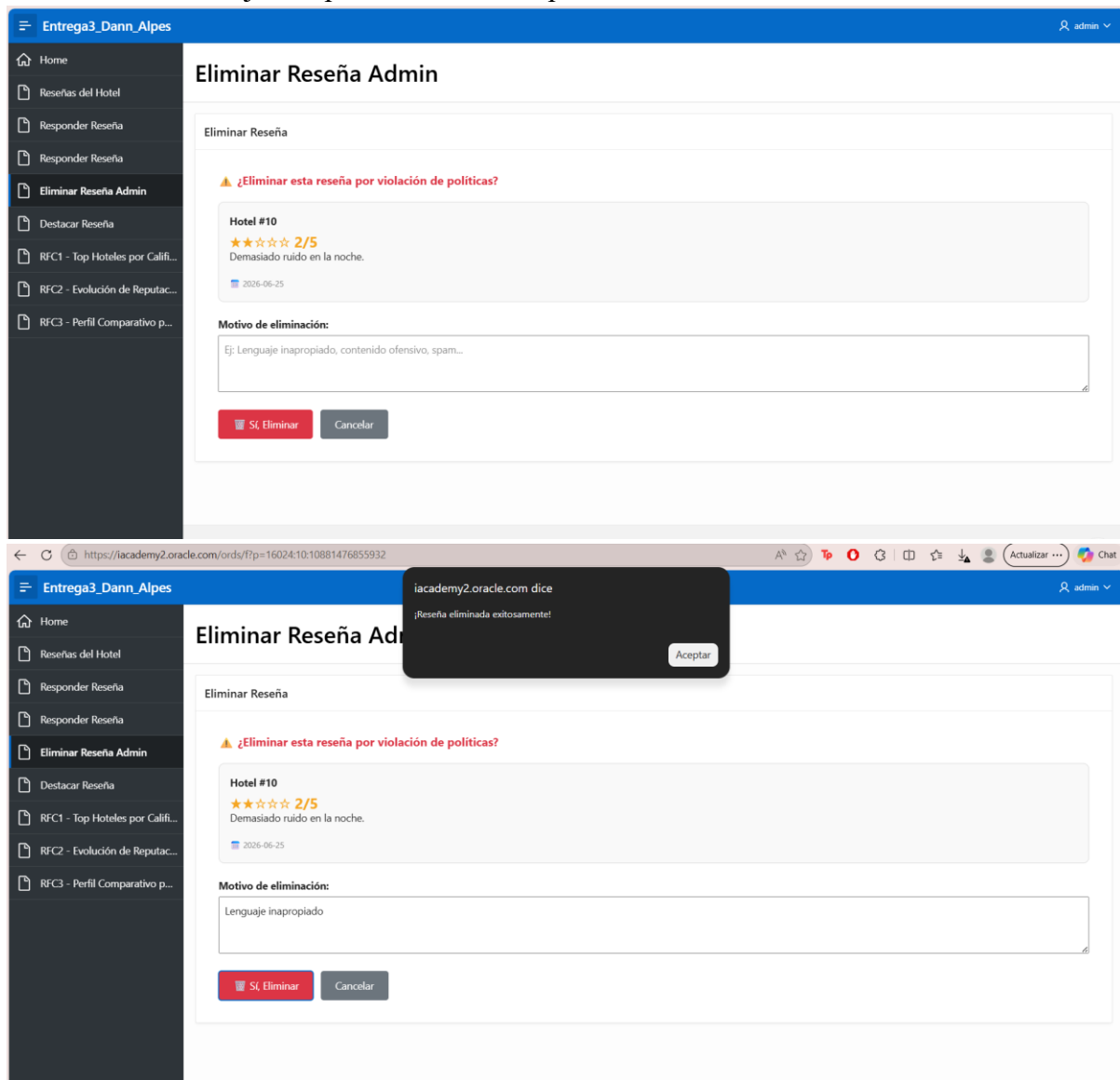
5.8 RF8 – Eliminar Reseña (Administrador)

Prueba realizada:

- Autenticado como ADMIN, desde el Panel se hizo clic en "Eliminar" sobre una reseña
- Se escribió un motivo: "Lenguaje inapropiado"
- Se confirmó la eliminación

Resultado:

- El sistema mostró "¡Reseña eliminada exitosamente!"
- La reseña dejó de aparecer en la vista pública



Validación:

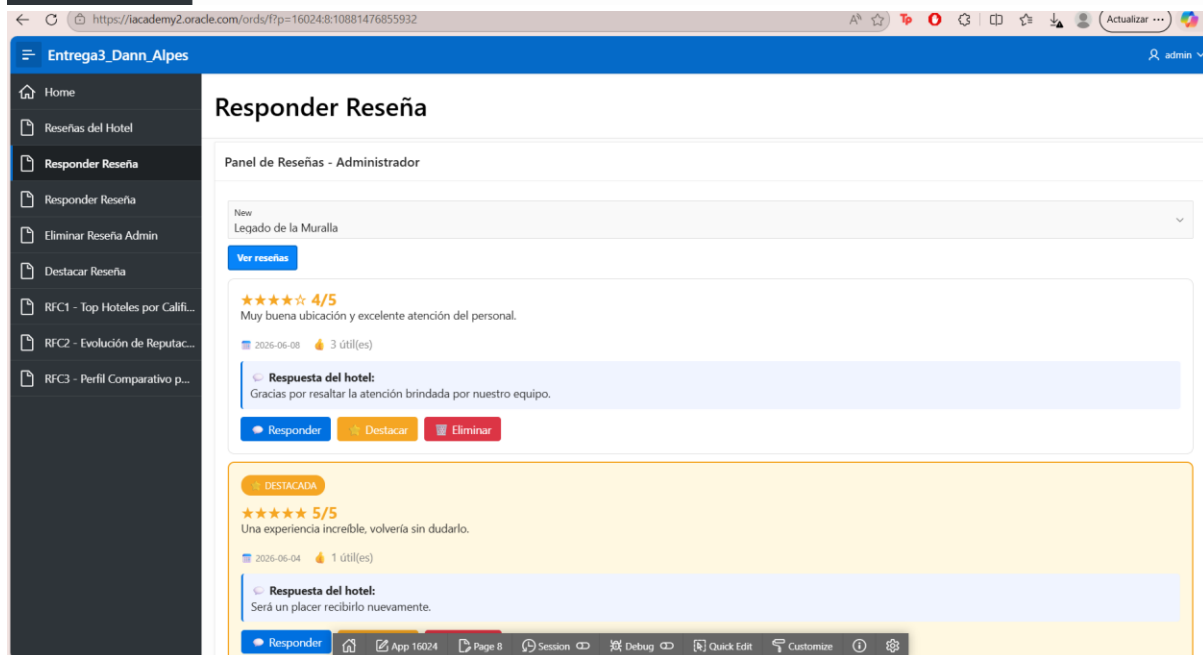
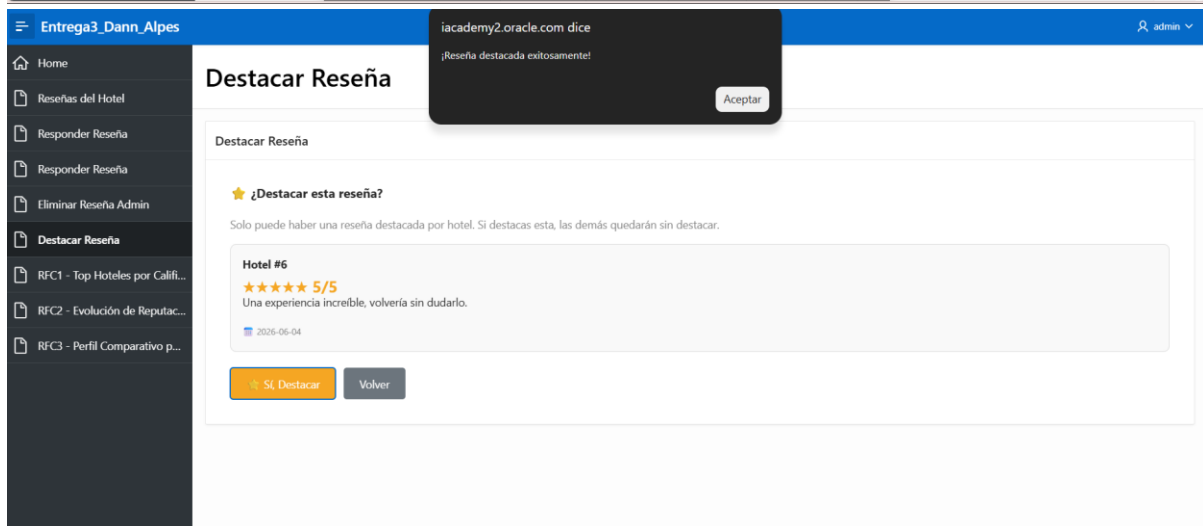
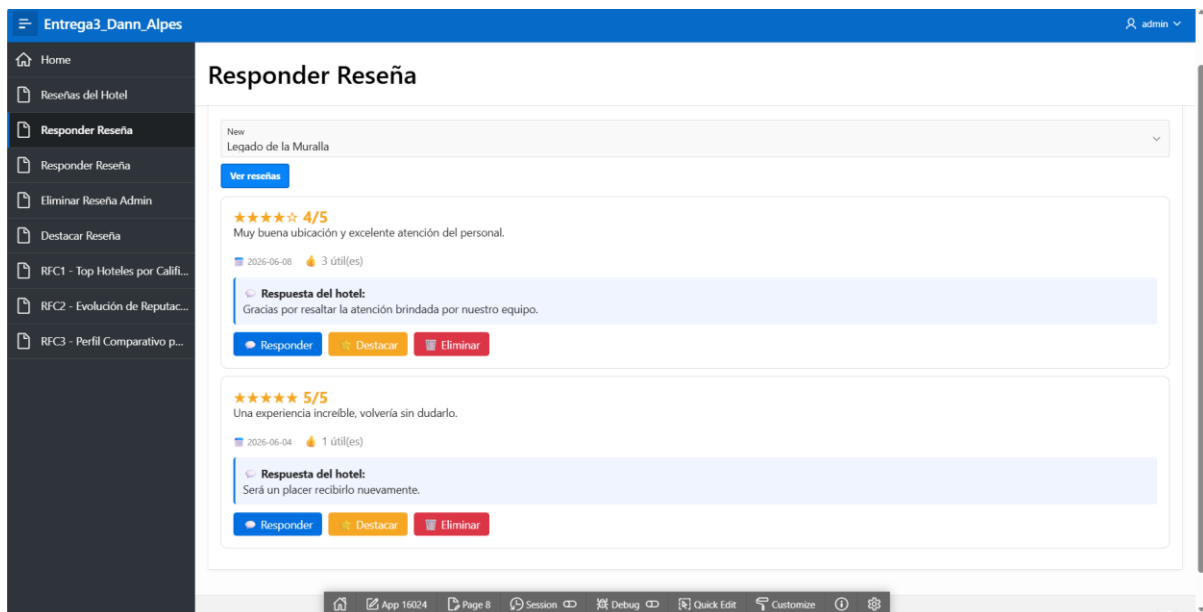
- En MongoDB se verificó: estado: "ELIMINADA", eliminada_por: <usuario_admin>, motivo_eliminacion: "Lenguaje inapropiado"
- El cliente dueño de la reseña puede verla en su historial con el badge de eliminada
- La validación obligatoria del motivo funcionó: al intentar eliminar sin motivo, el sistema mostró alerta de error

5.9 RF9 – Destacar Reseña**Prueba realizada:**

- Autenticado como ADMIN, desde el Panel se hizo clic en "Destacar" sobre una reseña
- Se confirmó la acción

Resultado:

- El sistema mostró "¡Reseña destacada exitosamente!"
- Al regresar al Panel, la reseña apareció con borde amarillo oscuro y badge "DESTACADA"



Validación:

- En MongoDB se verificó que el campo destacada: true se asignó solo a la reseña seleccionada
- Las demás reseñas del mismo hotel quedaron con destacada: false, cumpliendo la regla de "máximo una reseña destacada por hotel"
- Al destacar una segunda reseña, la anterior automáticamente quedó sin destacar

6. Escenarios de Prueba sobre el Esquema de Validación

6.1 Validación de esquema MongoDB para RF1 (Crear Reseña)

Objetivo: Verificar que el esquema de validación de la colección resenas impida insertar documentos malformados.

Script ejecutado en MongoDB Shell:

```
db.resenas.insertOne({
  descripcion: "Reseña sin id_hotel ni cliente"
})
```

Resultado esperado: Error de validación Document failed validation indicando que faltan los campos obligatorios id_hotel, documento_cliente, id_reserva, puntuacion_estrellas.

```
> db.resenas.insertOne({
  descripcion: "Reseña sin id_hotel ni cliente"
})
MongoServerError: Document failed validation
```

```
db.resenas.insertOne({
  id_reseña: NumberLong(99999999),
  id_hotel: 1,
  documento_cliente: NumberLong(2843140779),
  id_reserva: 1,
  puntuacion_estrellas: 10,
  descripcion: "Calificación inválida",
  estado: "publicada",
  destacada: false,
  fecha: new Date(),
  hora: "12:00"
})
```

Resultado esperado: Error de validación porque puntuacion_estrellas debe estar entre 1 y 5.

```
> db.resenas.insertOne({
  id_reseña: NumberLong(99999999),
  id_hotel: 1,
  documento_cliente: NumberLong(2843140779),
  id_reserva: 1,
  puntuacion_estrellas: 10,
  descripcion: "Calificación inválida",
  estado: "publicada",
  destacada: false,
  fecha: new Date(),
  hora: "12:00"
})
< Warning: NumberLong: specifying a number as argument is deprecated and may lead to loss of precision, pass a string instead
< Warning: NumberLong: specifying a number as argument is deprecated and may lead to loss of precision, pass a string instead
❗ • MongoServerError: Document failed validation
```

```
db.resenas.insertOne({
  id_reseña: NumberLong(99999998),
  id_hotel: 1,
  documento_cliente: NumberLong(2843140779),
  id_reserva: 2,
  puntuacion_estrellas: 4,
  descripcion: "Estado no permitido",
  estado: "INVENTADO",
  destacada: false,
  fecha: new Date(),
  hora: "12:00"
})
```

Resultado esperado: Error de validación porque estado solo acepta "publicada" o "ELIMINADA".

```
> db.resenas.insertOne({
  id_reseña: NumberLong(99999998),
  id_hotel: 1,
  documento_cliente: NumberLong(2843140779),
  id_reserva: 2,
  puntuacion_estrellas: 4,
  descripcion: "Estado no permitido",
  estado: "INVENTADO",
  destacada: false,
  fecha: new Date(),
  hora: "12:00"
})
< Warning: NumberLong: specifying a number as argument is deprecated and may lead to loss of precision, pass a string instead
< Warning: NumberLong: specifying a number as argument is deprecated and may lead to loss of precision, pass a string instead
❗ • MongoServerError: Document failed validation
```

6.2 Validación de esquema MongoDB para RF2 (Editar Reseña)

Script ejecutado en MongoDB Shell:

```
db.resenas.updateOne(  
  { _id: ObjectId("6a07b3f0e18f377ff062b832") },  
  { $set: { puntuacion_estrellas: 0 } }  
)
```

Resultado esperado: Error de validación porque la calificación debe ser entre 1 y 5.

```
> db.resenas.updateOne( { _id: ObjectId('6a07b3f0e18f377ff062b832') }, { $set: { puntuacion_estrellas: 0 } } )  
✖ ▶ MongoServerError: Document failed validation
```

```
db.resenas.updateOne(  
  { _id: ObjectId("6a07b3f0e18f377ff062b832") },  
  { $set: { puntuacion_estrellas: "cinco" } }  
)
```

Resultado esperado: Error de validación porque puntuacion_estrellas debe ser de tipo numérico.

```
> db.resenas.updateOne(  
  { _id: ObjectId("6a07b3f0e18f377ff062b832") },  
  { $set: { puntuacion_estrellas: "cinco" } }  
)  
✖ ▶ MongoServerError: Document failed validation
```

7. Balance del plan de Pruebas

Lo que se logró:

A nivel del esquema de validación en MongoDB:

- Se configuró correctamente el \$jsonSchema con validationLevel: "strict" y validationAction: "error" en la colección resenas
- Se validaron los casos de prueba para RF1 (inserción): documento sin campos obligatorios, calificación fuera de rango, y estado inválido. Todos arrojaron el error de validación esperado.
- Se validaron los casos de prueba para RF2 (actualización): calificación fuera de rango y tipo de dato incorrecto. Ambos arrojaron el error esperado.

A nivel de los requerimientos funcionales en APEX:

- Los 9 requerimientos funcionales (RF1-RF9) fueron probados visualmente desde la interfaz de APEX
- Los 3 requerimientos de consulta (RC1-RC3) fueron probados visualmente desde la interfaz de APEX
- Cada operación fue verificada en MongoDB Compass para confirmar la persistencia correcta de datos
- Se validó la integración Oracle con MongoDB usando los identificadores compartidos de hoteles, clientes y reservas
- Se confirmó el control de acceso por roles (CLIENTE y ADMIN) mediante Authorization Schemes
- Se verificaron las reglas de negocio críticas: unicidad de reseña por reserva, soft delete con trazabilidad, y máximo una reseña destacada por hotel

8. Población de Datos para Pruebas de RFC

Para garantizar la correcta ejecución de las consultas analíticas (RFC1, RFC2 y RFC3), se poblaron las colecciones de MongoDB con la siguiente cantidad de documentos:

Colección	Cantidad de documentos	Distribución
resenas	34	Distribuidas entre 20 hoteles, con fechas entre mayo y junio de 2026
votos	25+	Asociados a diversas reseñas
respuestas	30+	Una por cada reseña con respuesta administrativa

Los scripts de población se encuentran adjuntos en el repositorio:

- ResenasPoblacion.json
- Votos_De_UtilidadPoblacion.json
- Respuestas_AdministrativasPoblacion.json

Estos archivos fueron importados directamente mediante la opción "**Add Data > Import JSON File**" de MongoDB Compass, asegurando la consistencia de tipos Long para los identificadores numéricos largos (documento_cliente, id_reseña, etc.).

9. Problemas Identificados y Soluciones

Problema 1: Inconsistencia en formatos de fecha

Solución: Conversión uniforme mediante `$dateToString` en el pipeline

Problema 2: Diferencia en valores del campo estado

Solución: Uso de `$in` para aceptar valores como "PUBLICADA" y "publicada"

Problema 3: Parámetros vacíos desde APEX

Solución: Validación previa a la ejecución de la consulta y reglas en la base de datos de Mongo BD.

Problema 4: Ejecución del script antes de la carga del DOM

Solución: Verificación de la existencia del elemento canvas antes de renderizar el gráfico

10. Conclusión

El sistema desarrollado permite la integración efectiva entre una API REST, una base de datos NoSQL y una interfaz visual en APEX. Se implementaron correctamente los requerimientos funcionales de consulta, permitiendo visualizar datos analíticos en tiempo real mediante gráficos dinámicos e interacciones cliente-sistema/administrador-sistema.

Los módulos correspondientes a RFC1 y RFC2 fueron desarrollados, probados y validados satisfactoriamente. Al igual que los RF1 a RF9 de forma que todos los usuarios pueden interactuar con facilidad y de manera dinámica con la aplicación.

Por lo que, podemos decir que hemos cumplido satisfactoriamente con los requerimientos de nuestro cliente Dann Alpes.

Gracias.